

Padrões de Criação (Builder Pattern)

1. Escopo e Objetivo da Atividade

Esta atividade consiste na aplicação do padrão de projeto criacional **Builder**. O objetivo é refatorar a instanciação de objetos complexos, eliminando o anti-padrão conhecido como *Telescoping Constructor* (construtores telescópicos) e implementando uma Interface Fluente (*Fluent Interface*) que garanta a imutabilidade e a validação do estado do objeto antes de sua criação.

2. O Problema Arquitetural (Anti-Pattern)

Na engenharia de software, frequentemente deparamo-nos com classes que requerem dezenas de atributos para serem instanciadas, sendo muitos deles opcionais. A abordagem ingênua resulta na criação de múltiplos construtores sobrecarregados ou em chamadas confusas, onde o desenvolvedor é forçado a passar valores nulos (null) para parâmetros que não deseja utilizar.

Exemplo do Anti-padrão (A ser evitado):

```
// É impossível saber o que cada "null" ou "false" significa sem ler a documentação da classe.  
NotaFiscal nf = new NotaFiscal(1029, "10/10/2026", "Empresa X", "12.345.678/0001-99", 5000.00,  
250.00, null, null, false, true);
```

3. O Cenário Prático (Sistema de Faturamento ERP)

O Contexto: A equipe de desenvolvimento está construindo o módulo de faturamento de um sistema ERP. O núcleo deste módulo é a geração de Notas Fiscais (NF-e para produtos e NFS-e para serviços).

O Desafio: A entidade NotaFiscal é um objeto complexo. Ela possui atributos obrigatórios (Número, CNPJ, Razão Social) e atributos estritamente opcionais (Observações Adicionais, Transportadora, Endereço de Entrega Alternativo, Desconto Condicional). Além disso, a regra de negócio dita que **uma Nota Fiscal não pode ser alterada após ser emitida** (o objeto deve ser imutável).

4. Instruções de Execução

Etapa 1: Definição da Entidade Imutável (Product)

- **Tarefa:** Criar a classe NotaFiscal contendo pelo menos 8 atributos (misturando obrigatórios e opcionais).
- **Restrição Arquitetural:** A classe **não pode** ter métodos **Setters** públicos. O construtor da classe deve ser privado (private), impedindo que outros desenvolvedores instanciem a

classe diretamente com a palavra reservada `new`.

Etapa 2: Implementação do Padrão Builder

- **Tarefa:** Criar a classe `NotaFiscalBuilder`. Ela será a única responsável por coletar os dados e construir a `NotaFiscal`.
- **Interface Fluente:** Os métodos do *Builder* devem retornar a própria instância do *Builder* (ex: `return this;`), permitindo o encadeamento de chamadas. Os métodos devem ter nomes semânticos (ex: `comCnpj()`, `comItem()`, `comObservacoes()`).

Etapa 3: Validação e Construção (O método `build()`)

- **Tarefa:** Implementar o método final `.build()` dentro do *Builder*.
- **Regra de Negócio:** Antes de invocar o construtor privado da `NotaFiscal` e retornar o objeto pronto, o método `build()` deve executar validações matemáticas e lógicas.
 - *Exemplo de Validação exigida:* Se o CNPJ não tiver sido informado, ou se o valor total dos itens for menor ou igual a zero, o método `build()` deve lançar uma exceção formal (ex: `IllegalStateException`).